

HANS YANG

Toronto, ON 📞 (647) 862-5138

✉️ hans.yangchenghao@gmail.com

🐙 [GitHub](#)

🌐 [LinkedIn](#)

📁 [Portfolio](#)

Education

Wilfrid Laurier University

Sep. 2024 – Present

Bachelor of Science (Computer Science)

Major GPA: 11.89/12 (89%)

Relevant Courses: Algorithm Design & Analysis, Calculus I & II, Computer Networks, Databases I, Software Engineering, Data Structures I & II, Introduction to Microprocessors, Discrete Mathematics, Object Oriented Programming, Probability I

Technical Skills

Languages: C, C++, HTML/CSS, Go, Java, JavaScript, Lua Python, R, SQL, TypeScript, VBA

Software & Tech: AWS Lambda, AWS S3, Azure, Docker, Excel/Google Sheets, Git, GitHub, Matplotlib, MongoDB, MySQL, Node.js, NumPy, Pandas, Redis, Vercel, Vite

Work Experience

Me & You Studios

2026

Fullstack Gameplay Programmer, LiveOps

Virtual

- Engineered backend systems for player data persistence and implemented behavior trees for NPC decision-making, delivering new gameplay features coinciding with a **peak of 10,000+ concurrent users**.
- Developed frontend UI and VFX, contributing to the player-facing experience for a title that **scaled to 10K+ CCU**.
- Sourced, vetted, and onboarded external contractors, negotiating pricing and scope to align deliverables with project needs and budget.
- Shipped content under tight deadlines with unit and integration test coverage, **boosting D1 retention by over 10%**.
- Leveraged AI to improve documentation and navigate a complex legacy codebase, creating a more productive and maintainable development environment.

Projects

🔗 AI Flashcard Study Tool | *AWS Lambda, AWS S3, Docker, Gemini API, Go, Node.js, Typescript* 2026 – Current

- Designed a full-stack serverless AI document-processing pipeline using **AWS Lambda, AWS S3, and Docker** to transform uploaded files into structured flashcard data through automated text extraction and generation workflows.
- Developed backend services using **Go** and client integrations with **TypeScript/Node.js**, establishing an end-to-end architecture for document ingestion, AI processing, and user-facing content delivery.

🔗 Visulie | *Lua, Roblox Studio, State/Data-Driven Architecture*

2021 – Current

- Designed and shipped a full-stack authoring tool with a custom editor, versioned data format, and a schema covering 900+ property/instance types, replacing thousands of lines of manual scripting with a declarative, data-driven workflow, sustained across 15+ releases as a commercial product with **150+ sales and \$1.5K+ CAD revenue**.
- Actively ranked #1 on Google Search**, increasing plugin visibility and outreach through marketplace optimization and community engagement.
- Iterated on a customer-facing product, leveraging user feedback, feature requests, and bug reports to drive continuous improvements and deliver a more robust user experience.

🔗 FPS | *Lua, Roblox Studio*

2023

- Designed and shipped a full-stack interactive application with state-driven progression systems and a custom in-app economy, scaling to 100K+ users and 100+ concurrent sessions through iterative performance tuning and UX-driven front-end design.
- Featured in the official industry year-in-review** for applying parallel/concurrent execution techniques Lua and Canvas Groups with awareness of parallel execution constraints to improve performance and rendering throughput.

🔗 Pocket Rift | *Lua, Roblox Studio, State Machine/Event/Authoritative Server Architecture*

2019 – Current

- Implemented a product analytics pipeline to track user funnels and onboarding behavior, enabling data-driven iteration that improved conversion rates by over 185%.
- Designed a distributed client-server networking architecture with client prediction, state synchronization, and delta compression, **reducing latency and network bandwidth usage by 95% over baseline implementations**.
- Built reusable internal frameworks, including an in-memory caching system for session data and a reactive state management system, reducing system coupling and enabling scalable client-server architecture.

🔗 SPY Trading Algorithm | *Python, Matplotlib, yfinance*

Jan. 2026

- Designed and implemented a Python-based systematic trading strategy for SPY, backtested on historical market data with performance analytics, **outperforming buy-and-hold by approximately 54%**.
- Analyzed performance differences between a rule-based trading approach and buy-and-hold, identifying limitations such as sensitivity to market conditions and transaction assumptions.